Chiwire Maps 5: Maintenance

For whoever keeps the device working: how the project is organised, what to update and when, and how to find problems.

Contents

1. Project structure
2. Updating the map app on the card
3. Updating the firmware and its libraries
4. Updating the magnetic model
5. Refreshing the map tiles
6. Refreshing the search data
7. Running the tests
8. Debugging
9. Known limits and open work

Project structure

Path	What it is
`firmware/src/main.cpp`	Start-up, Wi-Fi network, GPS reading, screen, button, LED
`firmware/include/routes.h`	The web server's addresses: the map, `/location`, `/files`, `/download`, `/upload`, `/storage`, the tiles, the static files
`firmware/include/index.h`, `files.h`	The map page and Files page shells (a change needs a reflash)
`firmware/include/pathutil.h`	Pure helpers for paths, file names and sizes, tested on the PC
`firmware/include/Middlewares.h`, `MiddlewareSystem.h`	Request logging and the low-memory guard
`firmware/include/StatusLed.h`	The RGB LED driver
`firmware/test/`	Unit tests for `pathutil.h` (run on the PC)
`sdCard/nav.js`	The map app: layers, grid, panel, go-to, marks, search panels
`sdCard/geo.js`	UTM, grid convergence, magnetic declination, sun times, coordinate parsing

Path	What it is
`sdCard/marks.js`, `search.js`	The marks/routes data model, and the search logic
`sdCard/wmm.js`	The World Magnetic Model coefficients
`tileDownloader/spain_tiles.py`	The IGN tile downloader, with its plan and area files
`tools/build_search.py`	Builds the search data from OpenStreetMap
`tools/make_wmm_js.py`, `make_icon.py`, `make_guides.sh`, `card_copy_progress.py`	Small helpers: the magnetic model, the icon, the guides, the copy progress
`tests/`	Web, browser end-to-end, downloader and search builder tests
`docs/`	The guides and the work plan

The design rule: the dongle only stores and sends files. The maths, the drawing and the search run in the phone's browser.

Updating the map app on the card

The map app (`sdCard/*.js`) lives on the card, so most changes need no reflash. After any change, copy **all** the app files, not only the changed ones:

```
cp sdCard/{*.js,favicon.ico,icon-192.png} /mnt/beacon/ && sync
```

A stale file on the card (for example an old `nav.js` next to a new `search.js`) can hide features without any error. The phone re-checks the app files on every visit (`Cache-Control: no-cache`), so a reload picks up the new version.

Updating the firmware and its libraries

The firmware uses the Arduino-ESP32 core 2.0.x and these libraries (`firmware/platformio.ini`):

Library	Notes before updating
TFT_eSPI	The screen settings are in `build_flags` (`USER_SETUP_LOADED`). `USE_HSPI_PORT` is required on the ESP32-S3; without it the dongle crashes at start-up
ESPAsyncWebServer and AsyncTCP (esp32async)	The original `me-no-dev` versions were removed from the registry. The newer API returns `const` headers and parameters, which `Middlewares.h` already uses
TinyGPSPlus, ArduinoJson, QRCode	Stable

Rules:

- **Update one library at a time,** then build, flash and test on the device.
- **Prefer versions that still support core 2.x.** Moving to core 3.x is a bigger change; test it on its own.
- **exFAT stays unsupported** in this core build (`FF_FS_EXFAT 0`): the card must stay FAT32.
- After `pio run`, check the memory use (about RAM 14.0%, Flash 29.9% today). The dongle has no PSRAM; free heap while running is about 160–170 KB.

Updating the magnetic model

The World Magnetic Model 2025 is valid from 2025.0 to 2030.0. After that, the north box shows "EXPIRED", and the declination slowly becomes wrong.

When NOAA and BGS release WMM2030 (expected in late 2029):

1. Download the coefficient file (`WMM.COF`) from the NOAA World Magnetic Model page.
2. Convert it: `python3 tools/make_wmm_js.py path/to/WMM.COF sdCard/wmm.js`
3. Update the test values: replace `tests/web/fixtures/WMM2025_TestValues.txt` with the new model's official test values, and adjust the test that reads them.
4. Run the web tests (see below); they must pass against the official values.
5. Copy `sdCard/wmm.js` to the card.

Refreshing the map tiles

IGN updates the MTN, PNOA and Base maps from time to time. The downloader skips tiles already on disk, so to refresh an area:

1. Delete that area's tiles from `sdCard/tiles/<layer>/` (or download into a new folder).
2. Run the download again with the same command as in guide 1; it fetches only what's missing.
3. Copy to the card with the same rsync command; it copies only files that changed.

What to expect from IGN, measured during the full download:

- **No published speed limit.** 24 tiles per second ran clean. The adaptive mode halves the speed when more than 1% of requests in a 2-minute window fail, then climbs back to 90% of it. It stops with exit code 3 if most requests fail even at the lowest speed: that could be a block, so wait before trying again.
- **502 Bad Gateway for single tiles,** at any speed, about 0.01% of tiles. The same tile usually

works minutes or hours later.

- **403 AccessDenied** from the vector server for tiles that don't exist (open sea). The downloader counts them as missing, not failed.
- **Our own network errors** (Wi-Fi drops, waking from sleep) pause the download without changing the speed.
- Every window is logged as one row in `download_metrics.csv` in the output folder. Failed tiles are listed in `failed.txt`; `python3 spain_tiles.py retry` fetches them again.

Never use proxies or other tricks to go around IGN's limits.

Refreshing the search data

OpenStreetMap changes every day. To refresh the search (a few times a year is plenty):

1. Download the latest `spain-latest.osm.pbf` from Geofabrik.
2. Rebuild: `tools/.venv/bin/python tools/build_search.py tools/data/spain-latest.osm.pbf sdCard/search` (about 5 minutes; needs about 7.4 GB of RAM).
3. Replace the card's `search/` folder: `rsync -rt --modify-window=1 --delete sdCard/search/ /mnt/beacon/search/`

The search logic is in two places that must agree: `tools/build_search.py` (on the PC) and `sdCard/search.js` (on the phone). The fixture `tests/search/fixtures/normalize_cases.json` is checked by both test suites.

Running the tests

From the repository root:

Suite	Command	Needs
Web app	`node --test "tests/web/*.test.js"`	Node.js
Browser end-to-end	`cd tests && node --test "e2e/*.test.js"`	Node.js, Chrome or Chromium, Python 3
Downloader	`python3 -m unittest discover -s tests/downloader`	Python 3
Search builder	`tools/.venv/bin/python -m unittest discover -s tests/search`	the `tools/.venv` with pyosmium
Firmware helpers	`cd firmware && ~/.platformio/penv/bin`	g++

Suite	Command	Needs
	`/pio test -e native`	
Firmware build	`cd firmware && ~/.platformio/penv/bin/pio run`	PlatformIO

The maths is tested against official reference values: the NOAA/USNO sun tables, GeographicLib for grid convergence, and the WMM test values. The end-to-end tests run the real map page in headless Chrome against `tests/e2e/fake_dongle.py`, a small server that behaves like the firmware's `routes.h`. When you change `routes.h`, change the fake too.

Debugging

The serial log. With the dongle on USB, `~/.platformio/penv/bin/pio device monitor` shows:

- `SD card initialized.` or `Card Mount Failed` at start-up;
- `SD card: … bytes free of … (measured in N ms)`;
- `AP IP address: 192.168.4.1`;
- every request, and `GPS: N chars/s, N sentences ok, N failed checksum` every 10 seconds. The chars/s should match what the GPS sends (about 5000 for this module), and failed checksums should stay near 0.

Opening the monitor can reset the dongle on some systems. To read the log without a reset, start the monitor first, then plug in the dongle.

Crashes. The monitor decodes crash backtraces on its own (`monitor_filters = esp32_exception_decoder`). For a backtrace copied from elsewhere, use `addr2line` from the ESP32 toolchain with `firmware/.pio/build/ESP32-S3-DevKitC-1/firmware.elf`.

Memory. `http://192.168.4.1/diagnostics` returns the free heap and the uptime. Below 40 KB free, every request gets "Service Unavailable: Low Memory" (503) until memory recovers; the threshold is `criticalHeapThreshold` in `Middlewares.h`.

GPS. `http://192.168.4.1/location` returns the raw GPS state: `fix`, `latitude`, `longitude`, `altitude`, `satellites`, `hdop`, `speed_kmh`, `course`. `fix` is true only for a position less than 5 seconds old.

Known limits and open work

The device limits are listed in guide 1. The open device tests, measurements and ideas (marks saved on the card, search improvements, a Meshtastic mesh link) are in `TODO.md`.